TIC-TAC-TOE APPLICATION

JAVA PROJECT REPORT

SUBJECT: Java Programming

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY (CBIT)

Submitted By:

Name: Thota Ram Vinay

Roll No: 1601-25-771-116

Section: AID-2

Academic Year: 2026-2027

1. INTRODUCTION

The Neon Tic-Tac-Toe Application is a modern, graphical two-player desktop game developed in Java using the Swing framework. The application provides an engaging user experience through a stylish dark neon theme, smooth CardLayout-based screen navigation between the main menu and the game arena, and dynamic font-scaling pop animations when grid buttons are clicked. The game maintains standard rules for a 3x3 Tic-Tac-Toe grid, evaluating wins across rows, columns, and diagonals, and handling draws automatically.

2. OBJECTIVES

- To develop an interactive graphical Tic-Tac-Toe game using Java Swing.
- To apply Object-Oriented Programming principles and event-driven programming concepts.
- To implement seamless screen transitions using CardLayout.
- To enhance visual feedback with custom colors, fonts, and animation timers.
- To provide robust win, draw, and board-locking state checks during gameplay.

3. FEATURES OF THE APPLICATION

- Graphical user interface (GUI) built with Java Swing.
- Modern dark-mode theme with vibrant neon cyan and magenta accents.
- Interactive main menu with "Start Game" and "Exit" options.
- Dynamic pop-up tile animation on user clicks using javax.swing.Timer.
- Real-time turn tracking and status display (X's Turn, O's Turn, WINS, or DRAW).
- Board-locking mechanism upon game completion to prevent further moves.
- Quick options to start a new game or return to the main menu.

4. TECHNOLOGIES AND TOOLS USED

Technology/Tool	Purpose
Java	Programming language used to develop the application.
JDK 25	Java development kit used to compile and run the program.
Visual Studio Code / IDE	Code editor used for writing and running the Java files.
Java Swing (JFrame, JPanel, JButton, JLabel)	GUI component library used to build windows and controls.
CardLayout	Layout manager used to toggle between the menu and game views.
javax.swing.Timer	Used to create the smooth font-scaling click animation.

5. OBJECT-ORIENTED PROGRAMMING CONCEPTS USED

- 5.1 Classes and Inheritance: The main application class TicTacToe extends JFrame, inheriting window management behaviors and container properties.
- 5.2 Encapsulation: GUI components and game states are organized neatly as private/protected fields within the class structure.
- 5.3 Event-Driven Programming: Action listeners handle button clicks on both menu items and game board tiles asynchronously.

- 5.4 Reusability: Helper methods like button(String text) are utilized to generate styled buttons consistently across the application.

6. CLASS DESCRIPTION

Class	Purpose	Important Elements
TicTacToe	Controls the entire application, GUI layout, game logic, and animations.	JFrame, CardLayout, Timer, actionPerformed(), win(), draw(), reset()

7. WORKING OF THE APPLICATION

1. The program starts execution from the main() method, initializing the TicTacToe frame on the Event Dispatch Thread using SwingUtilities.invokeLater().
2. The CardLayout container displays the menu page (menu) featuring the title and start options.
3. Upon clicking **START GAME**, the card layout switches to the game page (game).
4. Player X makes the first move by clicking any empty grid tile.
5. A click triggers an action listener that sets the mark ("X" or "O"), updates colors, and executes a timer-based font pop animation.
6. The application checks for a winning line or a board draw after every move.
7. If a win or draw occurs, status messages update, and the board is locked.
8. Players can click **NEW GAME** to reset the board or **MENU** to return to the title screen.

8. ALGORITHM

1. Start.
2. Initialize frame, panels, layout managers, and game components.
3. Display the main menu (CardLayout -> "menu").
4. Read user interaction; if Exit is clicked, terminate the program.
5. If Start Game is clicked, switch view to the game arena (CardLayout -> "game").
6. Wait for user tile selection.
7. If the selected tile is empty and the board is active:
    - Assign current player mark ('X' or 'O').
    - Run font-scaling animation timer.
    - Check win conditions across rows, columns, and diagonals.
    - Check draw conditions (if all tiles are filled without a winner).
    - If game ends, display result and lock board; otherwise, switch turn and repeat.
8. Stop.

9. IMPLEMENTATION

The implementation is self-contained within a single source file (TicTacToe.java) utilizing standard Java Swing packages (javax.swing.*, java.awt.*).

```
import javax.swing.*;

import java.awt.*;

public class TicTacToe extends JFrame {

   JPanel main = new JPanel(new CardLayout());

   JPanel menu = new JPanel(new GridLayout(4, 1, 10, 15));

   JPanel game = new JPanel(new BorderLayout(8, 8));

   JPanel board = new JPanel(new GridLayout(3, 3, 8, 8));

  JButton[] b = new JButton[9];
```

```
 JLabel status = new JLabel("X's Turn", JLabel.CENTER);
  CardLayout cards = (CardLayout) main.getLayout();
  boolean xTurn = true;
 Color bg = new Color(20, 20, 35);
TicTacToe() {
      setTitle("Neon Tic Tac Toe");
      setSize(400, 500);
      setDefaultCloseOperation(EXIT_ON_CLOSE);
      setLocationRelativeTo(null);
      menu.setBackground(bg);
JLabel title = new JLabel("TIC TAC TOE", JLabel.CENTER);
      title.setFont(new Font("Arial", Font.BOLD, 32));
      title.setForeground(Color.CYAN);
      menu.add(title);
JButton start = button("START GAME");
      JButton exit = button("EXIT");
menu.add(start);
      menu.add(exit);
      menu.add(new JLabel("   X  vs  O", JLabel.CENTER));
start.addActionListener(e -> cards.show(main, "game"));
      exit.addActionListener(e -> System.exit(0));
      game.setBackground(bg);
      status.setFont(new Font("Arial", Font.BOLD, 25));
      status.setForeground(Color.CYAN);
      game.add(status, BorderLayout.NORTH);
board.setBackground(bg);
for (int i = 0; i < 9; i++) {
         b[i] = button("");
```

```
        b[i].setFont(new Font("Arial", Font.BOLD, 40));
        board.add(b[i]);
b[i].addActionListener(e -> {
            JButton btn = (JButton) e.getSource();
            if (!btn.isEnabled() || !btn.getText().isEmpty())
              return;
String mark = xTurn ? "X" : "O";
            btn.setText(mark);
            btn.setForeground(xTurn ? Color.CYAN : Color.MAGENTA);
            btn.setFont(new Font("Arial", Font.BOLD, 10));
            final int[] size = {10};
            Timer t = new Timer(15, null);
            t.addActionListener(a -> {
              size[0] += 5;
              btn.setFont(new Font("Arial", Font.BOLD, size[0]));
              if (size[0] >= 40) t.stop();
            });
            t.start();
 if (win()) {
              status.setText(mark + " WINS!");
              status.setForeground(Color.GREEN);
              lockBoard();
            } else if (draw()) {
              status.setText("IT'S A DRAW!");
              lockBoard();
            } else {
              xTurn = !xTurn;
              status.setText(xTurn ? "X's Turn" : "O's Turn");
```

```
            status.setForeground(
              xTurn ? Color.CYAN : Color.MAGENTA);
          }
        });
      }
game.add(board, BorderLayout.CENTER);
JPanel buttons = new JPanel(new GridLayout(1, 2, 8, 8));
      buttons.setBackground(bg);
JButton restart = button("NEW GAME");
      JButton back = button("MENU");
restart.addActionListener(e -> reset());
      back.addActionListener(e -> {
        reset();
        cards.show(main, "menu");
      });
buttons.add(restart);
      buttons.add(back);
      game.add(buttons, BorderLayout.SOUTH);
main.add(menu, "menu");
      main.add(game, "game");
      add(main);
cards.show(main, "menu");
      setVisible(true);
  }
JButton button(String text) {
      JButton btn = new JButton(text);
      btn.setFont(new Font("Arial", Font.BOLD, 18));
      btn.setBackground(new Color(40, 40, 70));
```

```
        btn.setForeground(Color.CYAN);
        btn.setFocusPainted(false);
        btn.setBorderPainted(false);
        return btn;
    }
boolean win() {
        int[][] lines = {
            {0,1,2}, {3,4,5}, {6,7,8},
            {0,3,6}, {1,4,7}, {2,5,8},
            {0,4,8}, {2,4,6}
        };
for (int[] l : lines) {
            String s = b[l[0]].getText();
            if (!s.isEmpty() && s.equals(b[l[1]].getText())
                && s.equals(b[l[2]].getText()))
                return true;
        }
        return false;
    }
boolean draw() {
        for (JButton btn : b)
            if (btn.getText().isEmpty()) return false;
        return true;
    }
void lockBoard() {
        for (JButton btn : b) btn.setEnabled(false);
    }
```

```
  void reset() {
    for (JButton btn : b) {
      btn.setText("");
      btn.setEnabled(true);
      btn.setFont(new Font("Arial", Font.BOLD, 40));
    }
    xTurn = true;
    status.setText("X's Turn");
    status.setForeground(Color.CYAN);
  }
public static void main(String[] args) {
    SwingUtilities.invokeLater(TicTacToe::new);
  }
}
```

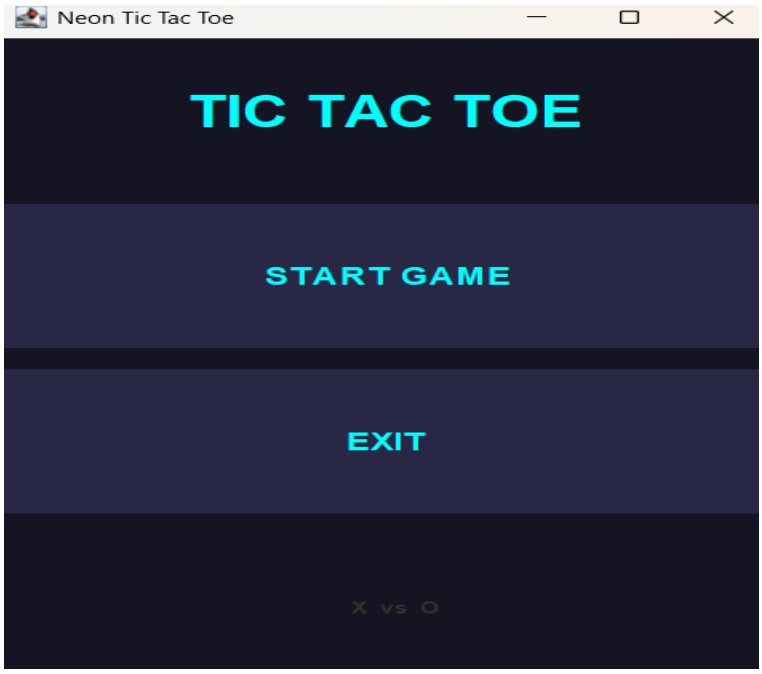
10. TESTING

Test Case	Input / Action	Expected Result	Status
1	Click **START GAME** on Menu	Application switches to the 3x3 game board arena view.	**PASS**
2	Click an empty grid tile	Tile displays the active player's mark with a smooth pop animation.	**PASS**
3	Click an already occupied tile	The click is ignored; the mark remains unchanged.	**PASS**
4	Align 3 matching symbols	Status updates with winner announcement and board locks.	**PASS**

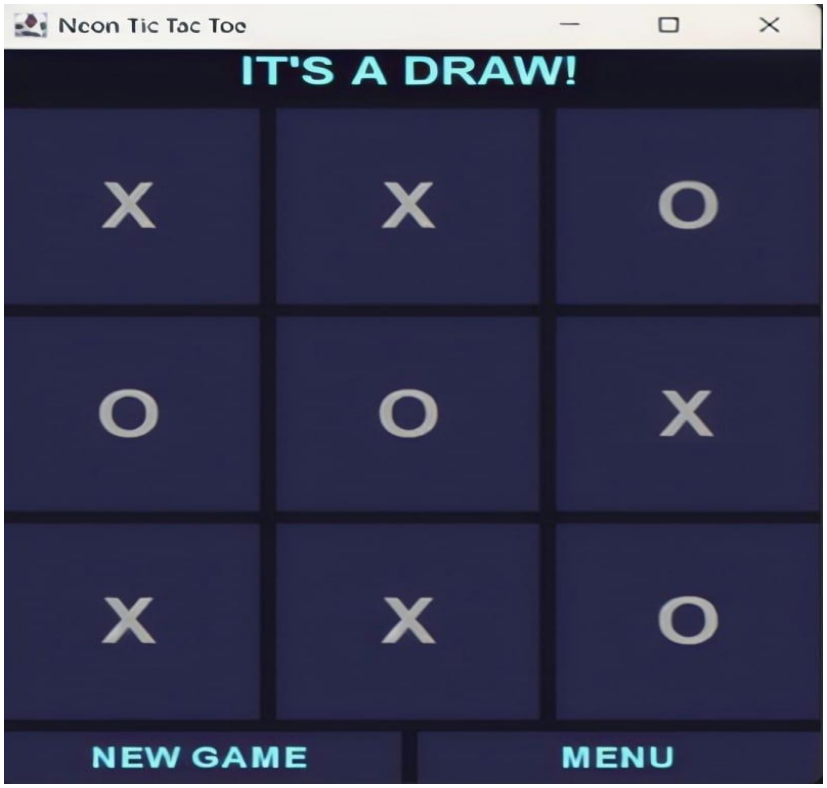
Test Case	Input / Action	Expected Result	Status
5	Fill all 9 tiles without a winner	Status updates to declare a draw and locks the board.	**PASS**
6	Click **NEW GAME / MENU**	Board resets successfully or returns user to the main menu view.	**PASS**

11.OUTPUT SCREENSHOTS

11.1 Main Menu ScreenShot:



11.2 PlayerVsPlayer(TwoPlayer)



12. ADVANTAGES

- Clean graphical interface with modern dark neon visuals.
- Smooth built-in timer animations enhance user engagement.
- Lightweight structure contained entirely within a single class file.
- Instant win and draw evaluation.
- Easy to compile and run across platforms supporting Java Swing.

13. LIMITATIONS

- The application currently supports local 2-player mode only (no AI opponent).
- Score statistics across multiple rounds are not permanently stored.
- Fixed grid size of 3x3.

14.FUTURE ENHANCEMENTS

- Integrate a Single Player mode against an AI opponent using the Minimax algorithm.
- Add persistent score tracking (Wins, Losses, Draws).
- Introduce custom player name inputs and sound effects.
- Support customizable grid sizes (e.g., 4x4 or 5x5).

15. CONCLUSION

The Neon Tic-Tac-Toe Application was successfully developed as a desktop application using Java Swing. The project effectively demonstrates core object-oriented principles, GUI layout management via CardLayout and GridLayout, action event handling, and custom timer animations. It fulfills all functional requirements for a robust 2-player board game while providing a polished, professional user interface.

16. REFERENCES

- Java Documentation - Oracle
- Swing API Specification and Tutorial Material
- Class notes and laboratory material
- Java Development Kit (JDK) documentation